

Embedded Security

Academic year 2026–2027

Attack The Vault

Write-up

Author
Chaimae AJNAIN

Teacher
Quentin DELHAYE

ECAM Brussels

October 22, 2026

Contents

1 Introduction **2**

2 Asset description **2**

3 First contact with the target **2**

4 Hardware setup **3**
4.1 Preparing the target . . . 3
4.2 Configuring the CW-Nano as high impedance . . . 3
4.3 Adapting the connections . . . 4

5 Vulnerability identification **4**

6 Attack tree **5**

7 Attack: description and implementation **5**
7.1 Measurement setup . . . 5
7.2 Candidate selection per position . . . 6
7.3 Choice of the averaging parameter . . . 6
7.4 Password reconstruction loop . . . 7

8 Results **7**
8.1 Reconstructed password . . . 7
8.2 Gap ratio per position . . . 7

9 Vulnerability severity assessment **8**

10 Counter-measures **8**

11 Conclusion **9**

1 Introduction

As part of the Embedded Security course, we were assigned a project called Attack The Vault!. The objective is to recover the password required to access a vault, represented by an Arduino Uno running dedicated firmware.

The vault contains a secret keyphrase that is protected by a password verification mechanism. The challenge is to gain access to the vault without prior knowledge of the password, by identifying and exploiting a vulnerability in the system.

This project aims to apply the concepts covered during the course to a practical security assessment. It involves analysing the target's behaviour, identifying potential vulnerabilities, implementing an attack, evaluating its severity, and proposing an appropriate countermeasure.

This report describes the attack, its results, the identified vulnerabilities, and the proposed countermeasures.

2 Asset description

The primary asset is the secret keyphrase stored in the Arduino Uno. Its confidentiality must be preserved to prevent unauthorized disclosure.

The system also exposes two secondary assets: the hash of the secret keyphrase and its associated salt. The hash is a cryptographic representation of the secret keyphrase, while the salt is an additional value used in the hashing process. Although the salt is not generally considered secret, both values are relevant to the security of the system. Their disclosure may reveal information about how the secret keyphrase is processed and protected. Depending on the hashing algorithm and its parameters, these values could potentially be used to attempt an offline recovery of the original secret.

The firmware is another important asset. Its confidentiality may be relevant because analysing its contents could reveal implementation details or potential vulnerabilities. Its integrity must also be preserved to ensure that the code executed by the Arduino has not been modified without authorization.

The attacker's primary objective is to recover the secret keyphrase without prior knowledge of it. This objective forms the root node of the attack tree presented in Section 6.

3 First contact with the target

The first step consisted in establishing a basic communication channel with the target in order to observe its behaviour. A direct cable connection was made between the Arduino Uno and a PC, and the serial output was monitored using PuTTY. On the default UART pins (digital pins 0 and 1, RX and TX respectively), the target replied with the message *"this is not the main entrance"*.

This response suggested that the standard UART interface did not provide access to the intended entry point of the vault, and that an alternative communication path might exist on the board. To investigate further, a USB-to-TTL adapter was connected to the Arduino, wiring the adapter's GND, TX and RX lines to the corresponding pins on the target. Reconnecting through pins 0 and 1 using this adapter reproduced the same rejection message, as expected since it merely provided an alternative means of reaching the same interface. This confirmed that pins 0 and 1, while a functional communication interface, did not give access to the intended entry point.

The remaining digital pins of the Arduino Uno were then probed manually, one pair at a time, by moving the TX/RX connections of the USB-to-TTL adapter across the available pins and observing the resulting output. Early attempts at several baud rates produced only garbled characters, until a baud rate of 9600 bps yielded a coherent, readable stream, confirming the correct serial configuration and narrowing down which pins were worth testing further. Following this process, pins 10 and 11 were identified as the actual RX/TX pair used by the target: connecting to these pins at 9600 baud produced an *"Enter password"* prompt instead of the rejection message, revealing the existence of a second, hidden UART interface, the true "main entrance" to the vault.

While effective, this manual, pin-by-pin and baud-rate-by-baud-rate search was time-consuming and offers limited reproducibility. A more systematic approach would have been to connect a logic analyzer to all candidate pins simultaneously and capture their activity while interacting with the target. The idle-high behaviour and characteristic start/data-bit transitions of an active UART line would then be directly visible in the capture, allowing both the correct RX/TX pair and the baud rate to be determined in a single step, rather than through exhaustive manual trial and error.

4 Hardware setup

4.1 Preparing the target

The hardware setup was based on the *Connecting to an External Target* guide provided in the professor's course wiki.[1] The guide describes how to prepare an Arduino-based target for power analysis with the ChipWhisperer-Nano. It recommends removing the ATmega328P from the Arduino Uno and mounting it on a breadboard to provide direct access to the microcontroller's power supply and relevant signals. This makes it possible to modify the power supply, introduce a measurement shunt and connect the ChipWhisperer-Nano without relying on the original board's layout.

Following these instructions, the ATmega328P was placed on a breadboard. The Arduino board remained connected to provide the 5 V supply, ground and reset signals. Pins 9 and 10 of the empty socket on the Arduino Uno were connected to pins 9 and 10 of the ATmega328P on the breadboard, corresponding to the OSC1 and OSC2 signals. This preserved the connection to the Arduino Uno's existing clock circuit without requiring a separate crystal.

The setup also included the RC filter and shunt resistor recommended in the guide to facilitate power consumption measurements. The RC filter smooths the power supply to reduce external noise, while the shunt resistor converts the target's current consumption into a measurable voltage drop. This voltage drop was probed at ATmega328P pin 7 (VCC) and connected to the CW-Nano's dedicated MEASURE port. In addition, the `measure` and `glitch` jumper pads on the ChipWhisperer-Nano were desoldered to reduce measurement noise.

4.2 Configuring the CW-Nano as high impedance

The ChipWhisperer-Nano embeds its own STM32F0 microcontroller, normally used as a built-in training target. This microcontroller shares its GPIOs with the same 20-pin connector used to reach an external target, so without further configuration it could keep driving these lines and interfere electrically with the Arduino. To prevent this, the STM32F0's GPIOs were set to high impedance (tristate), using the `gpio-tristate` firmware provided in the course repository[2]: compiled for the CW-Nano platform and flashed onto the STM32F0 through the ChipWhisperer

[1] `https://github.com/parastuffs/Embedded-Security/wiki/Connecting-to-an-external-target`
[2] `https://github.com/parastuffs/Embedded-Security/blob/main/Project/gpio-tristate.c`

Python API, this firmware puts its GPIOs in a state where they no longer impose a voltage level, leaving the lines free for the external target alone.

4.3 Adapting the connections

Once the target was mounted on the breadboard, the connections had to be adapted to match the interfaces identified during the initial investigation. Using a multimeter in continuity mode, Arduino digital pins 10 and 11 were mapped to ATmega328P pins 16 and 17, respectively. These connections were used for UART communication with the ChipWhisperer-Nano at 9600 baud.

Arduino digital pin 13 was also traced to ATmega328P pin 19, corresponding to the PB5/SCK pin in the microcontroller's pinout. An important hint in the course wiki stated that Arduino pin 13 is pulled HIGH during password verification and remains LOW otherwise. This behaviour made pin 13 suitable as a trigger for synchronising power consumption measurements with the password-verification operation. This trigger signal (ATmega328P pin 19) was connected to CW-Nano pin 16 of its 20-pin connector (TIO4, its dedicated trigger input), as shown in Figure 1.

The ChipWhisperer-Nano was connected through its 20-pin header: CW-Nano pin 2 to GND, CW-Nano pin 8 to the target's 5 V reference, CW-Nano pin 10 (RX) to the target's UART TX signal on ATmega328P pin 17, and CW-Nano pin 12 (TX) to the target's UART RX signal on ATmega328P pin 16.

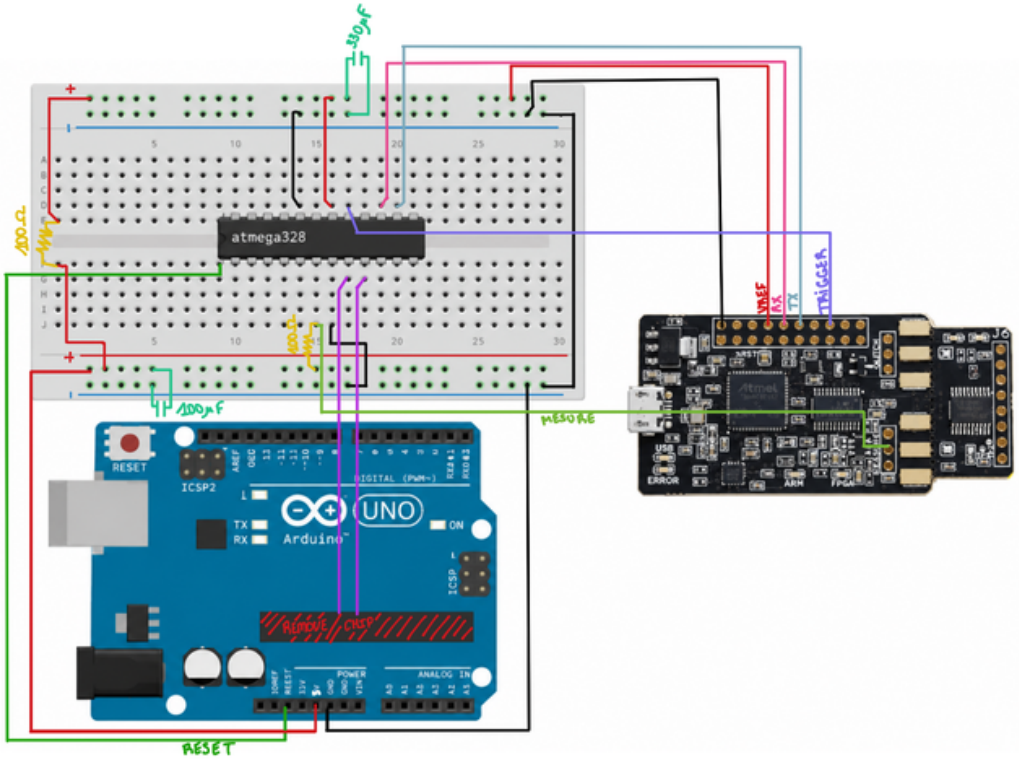


Figure 1: Final hardware setup.

5 Vulnerability identification

Most password verification routines compare the input character by character (or byte by byte) and stop as soon as a mismatch is found. As seen in the course, this is a well-known weakness: for instance, the C standard library's `strcmp` function behaves this way, and if used for a password check, it leaks information about the number of correct characters through its execution time. More generally, this behaviour also affects power consumption: an attacker measuring it

does not need to guess the whole password at once, as each character can instead be recovered independently, considerably reducing the search space.

Evidence of this behaviour on the target is given by the hint that Arduino pin 13 is pulled HIGH for the duration of the password verification and LOW otherwise, as discussed in Section 4. This confirms the existence of a precise, measurable time window corresponding to the comparison operation, which can be correlated with power consumption to recover the password one character at a time.

6 Attack tree

Figure 2 presents the attack tree built around the side-channel vulnerability described in Section 5, which is the one retained and exploited in this report. It follows the attacker/identification/exploitation/goal structure introduced during the course: the attacker's physical access to the target leads, through the identification steps described in Sections 3 and 4, to a single exploitation branch converging towards the goal, the disclosure of the secret keyphrase. The proposed counter-measure, detailed in Section 10, is also represented, mitigating this exploitation branch.

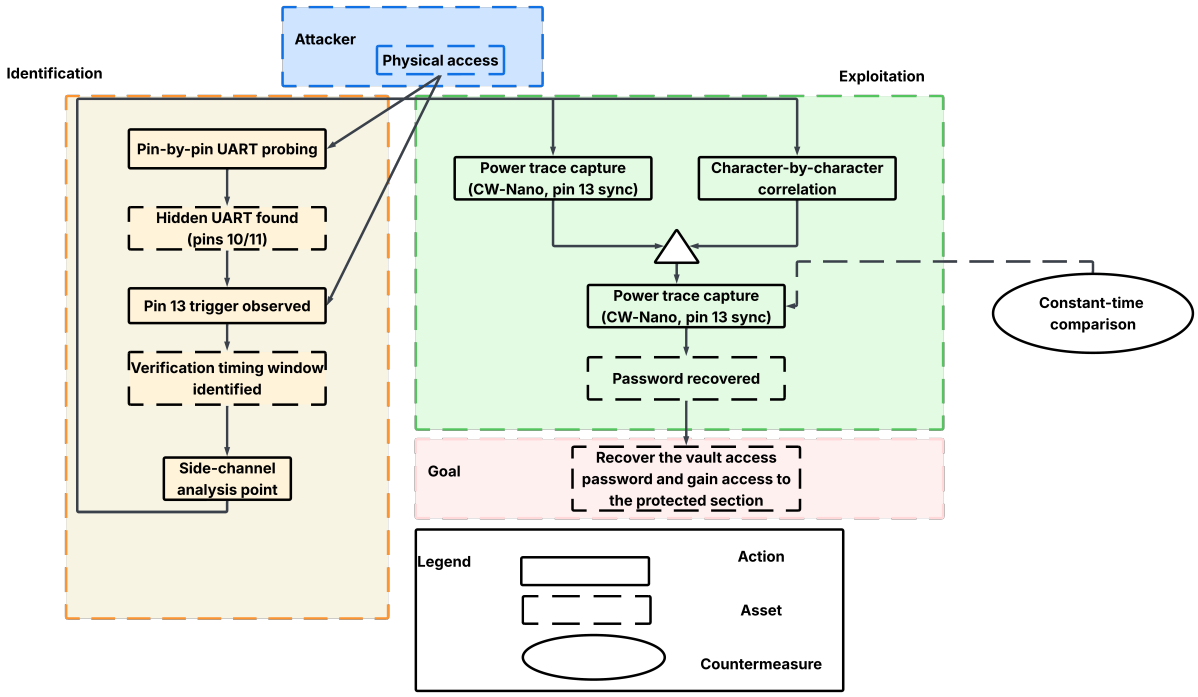


Figure 2: Attack tree of the vault, covering the side-channel vulnerability exploited as the primary attack.

7 Attack: description and implementation

The attack exploits the power side-channel vulnerability identified in Section 5: since the password comparison is performed character by character and diverges in behaviour as soon as a mismatch occurs, each position of the password can be recovered independently by comparing the power consumption induced by every candidate character.

7.1 Measurement setup

The ChipWhisperer-Nano measures power consumption indirectly, through the voltage drop across the shunt resistor on the ATmega328P's VCC line (Section 4). The ADC samples this volt-

age at a configured sampling frequency of 7.5 MHz, recording 5000 samples per trace (≈667 µs), triggered on TIO4 / Arduino pin 13 to align each trace on the password-verification window. This sampling frequency is a configuration parameter of the ChipWhisperer-Nano's acquisition system and does not necessarily match the ATmega328P's own clock, which, in this setup, retains the Arduino Uno's original 16 MHz clock (Section 4). For a given candidate password, n_{avg} traces are captured over UART and averaged sample-by-sample to reduce measurement noise while preserving the candidate's deterministic power signature; failed captures are discarded.

7.2 Candidate selection per position

For a known prefix, every character of the printable ASCII range (94 characters, `!` to `~`, as hinted in the course wiki) is appended in turn and an averaged trace is captured. A reference trace is computed by averaging across these 94 candidate traces, representing the typical behaviour when the character is wrong. Each candidate is then ranked by its Euclidean (L2) distance to this reference (chosen for its simplicity). The most divergent candidate is taken as correct, since a right character makes the verification continue further, altering its power signature compared to an immediate mismatch.

7.3 Choice of the averaging parameter

A convergence study on the first position (1000 raw traces, *gap ratio* between best and second-best candidate computed for $n \in \{25, ..., 1000\}$) showed the gap ratio rising sharply from ≈36% at $n = 25$ to ≈70% at $n = 250$, then flattening out: $n = 500$ gives essentially the same gap ratio (≈69.5%), and further increases up to $n = 1000$ only yield a marginal gain, reaching ≈73% (Figure 3).

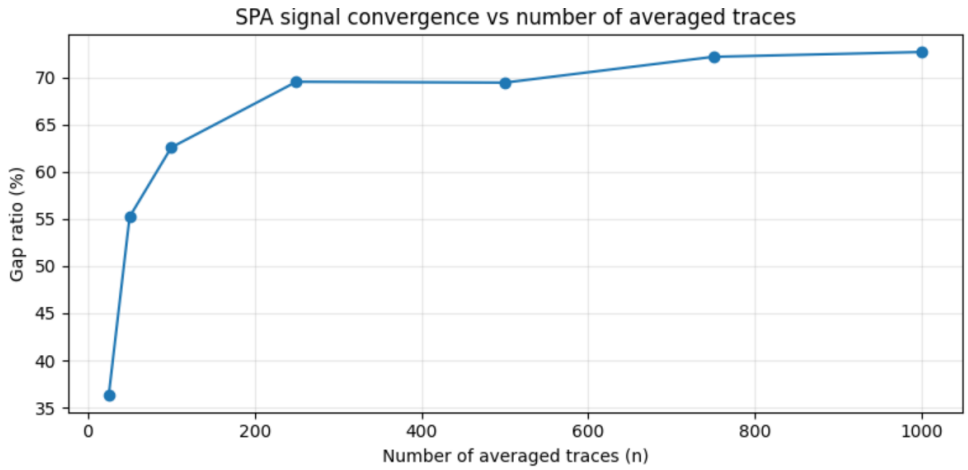


Figure 3: Convergence of the gap ratio (relative distance between the best and second-best candidate) as a function of the number of averaged traces, measured at the first password position against four decoy candidates. The curve rises sharply up to n=250 and flattens thereafter.

Capturing 250 traces per candidate at every position was however too slow in practice (94 × 500 traces per position). The reconstruction loop therefore used $n_{avg} = 50$, a deliberate speed/robustness trade-off: the gap ratio at that point (≈55%) is already well past the steep initial rise of the curve, even though convergence is only reached from $n \approx 250$ onwards.

A more robust variant was also implemented, repeating the analysis per position and selecting by majority vote, at the cost of proportionally more captures.

7.4 Password reconstruction loop

The password is built one character at a time: each position runs the candidate-selection step, and the resulting candidate password is verified directly against the vault, after flushing the UART buffer to prevent leftover responses accumulated during the candidate-selection captures from being mistaken for the verification reply. If the response no longer contains *"ACCESS DENIED"*, the password is found and the loop stops; otherwise the best candidate is appended and the loop continues, up to a safety length limit, with a secondary stop if the gap ratio falls below a fixed threshold.

At each position, the resulting candidate password is tested against the vault, but its *"ACCESS DENIED"* response is the same whether the prefix so far is correct (but incomplete) or actually wrong, it only turns positive once the complete password is correct. An incorrect character chosen at an intermediate position therefore cannot be distinguished from a correct-but-unfinished one, and would only surface once the full reconstruction fails or runs out of distinguishable candidates ($n_{avg} = 50$ proved sufficient to avoid this here).

8 Results

Applying the character-by-character power analysis described in Section 7, the full password was recovered one character at a time, each candidate being tested by averaging `n_avg = 50` power traces and selecting the character whose trace was closest, by L2 distance, to the reference traces.

8.1 Reconstructed password

The password was fully reconstructed after 8 positions:

`f7-@Jp0w`

Submitting this password to the vault returned the following confirmation:

```
Vault response for 'f7-@Jp0w': >> ACCESS GRANTED: running protected section
Here is your salt: 5B4C3C42
Here is your hash: DD1783FE7DC4418650E2C23B91ED4B2D2EAFBD84188C5C5307A8B5D694CDA9
```

This hash and salt, returned only after the correct password was submitted, serve as proof that the vault was successfully entered.

8.2 Gap ratio per position

The results are satisfactory overall: the gap ratio between the best and second-best candidate remained consistent with expectations throughout the reconstruction.

Position	Recovered character	Gap ratio
1	f	51.4%
2	7	52.5%
3	-	51.9%
4	@	47.3%
5	J	44.3%
6	p	56.0%
7	0	36.1%
8	w	47.0%

Based on the convergence study presented in Section 7, averaging 50 traces per candidate was expected to yield a gap ratio around 55%. The values observed during reconstruction mostly fall

close to, though slightly below, this expectation, clustering around 45–52%, with one position reaching the expected 56%. This is not surprising: the convergence study measured the gap ratio against only four fixed decoy candidates, whereas the actual reconstruction compares each captured trace against the full character set (ASCII 33–126). With close to a hundred competing candidates instead of four, it is statistically more likely that some second-best candidate happens to lie close to the best one by chance, which naturally lowers the gap ratio without indicating a less reliable measurement.

Position 7 shows a noticeably lower gap ratio (36.1%), and position 8 required averaging over only 44 valid traces instead of 50, due to six capture timeouts. Despite this reduced margin, both positions were still resolved correctly, confirming the robustness of the `n_avg = 50` trade-off chosen in Section 7.

9 Vulnerability severity assessment

Several frameworks exist to assess hardware security issues: the Common Vulnerability Scoring System (CVSS) scores how severe a known, exploitable vulnerability is, while the Common Weakness Scoring System (CWSS) scores general weaknesses that are not necessarily exploitable, and the Common Criteria Joint Interpretation Library (JIL) is used for formal hardware certification and attack-path rating. As the vulnerability identified in Section 5 was demonstrated in practice, it is assessed here using CVSS v4.0.

Vector: `CVSS:4.0/AV:P/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N/CR:H /MVC:H/AU:N/V:C/R`

Score: 5.1 (Medium).

The attack requires physical access to the target, since the microcontroller has to be removed and connected to dedicated measurement equipment to capture power traces. It cannot be performed remotely. Although the attack requires some knowledge of hardware and power analysis, it does not rely on bypassing any additional security mechanism, so the attack complexity is considered Low. No prior privileges or interaction with another user are required.

The attack allows the password to be recovered and access to the vault to be gained. The vault then returns the hash and salt of the secret keyphrase. Although the keyphrase itself was not recovered, the password protecting access to the vault was compromised. This is considered a High confidentiality impact, as it allows access to the protected section and could expose sensitive information. No impact on integrity or availability was demonstrated. Since the attack only affects the vault itself, the subsequent system impact metrics are set to None.

10 Counter-measures

The vulnerability identified in Section 5 stems from a password comparison that processes characters sequentially and stops as soon as a mismatch is found. This pattern is a well-documented weakness: the Common Weakness Enumeration entry CWE-1255 describes exactly this scenario, where a byte-by-byte comparison consumes a measurably different amount of power depending on whether each byte matches, allowing an attacker to recover the password one byte at a time through repeated measurements.[3]

The main countermeasure proposed is to replace the early-exit password comparison with a constant-time comparison. Instead of stopping at the first mismatching character, the implementation should process the entire password and accumulate the differences before making the

[3] `https://cwe.mitre.org/data/definitions/1255.html`

final decision. This prevents the execution time from directly revealing how many characters match the reference password and addresses the root cause of the identified vulnerability.

As a concrete example, CWE-1255 recommends replacing early-exit comparisons with an accumulator pattern such as `result |= (input[i] ^ reference[i])`.[4] The whole password is processed, and access is granted only if `result` is zero after the loop. Applying this approach to the vault's firmware would require no additional hardware.

However, constant-time execution does not guarantee constant power consumption. The same sequence of instructions is executed whatever the input, which removes the leakage exploited in this attack (the difference in duration and shape of the trace). A residual, data-dependent leakage nevertheless remains: the power drawn by CMOS logic depends on the values being manipulated, for instance their Hamming weight or the number of bits switching in a register. This leakage is much weaker and would likely require statistical techniques such as DPA or CPA, with many more traces, rather than the simple comparison of traces used here.

To address this residual leakage, a complementary hardware countermeasure is a current regulator or current equaliser, which keeps the power consumption seen from the outside approximately constant regardless of the operations performed. Such circuits are found *on-chip* in secure smart cards. In the threat model considered here, where the attacker has physical access to the target, a countermeasure placed outside the chip can be bypassed, for instance by measuring upstream of it, as was done when the ATmega328P was extracted from the Arduino board. This countermeasure is therefore only effective if integrated into the chip itself, which is not possible on an off-the-shelf ATmega328P.

Therefore, although these countermeasures are expected to reduce the leakage exploited by the attack, their effectiveness against power analysis should be experimentally evaluated on the target device. Since none of them was implemented or tested during this project, their effectiveness remains unverified.

11 Conclusion

This project set out to recover the password protecting the vault and, through it, the hash and salt revealing successful access. The side-channel vulnerability identified in Section 5, a direct instance of the comparison weakness described in the course and documented as CWE-1255, proved sufficient to achieve this: the full 8-character password was reconstructed one character at a time by measuring the target's power consumption, without ever needing to guess it outright.

What stands out most from this project is how much of the effort went into the hardware setup rather than the attack algorithm itself: finding the hidden UART interface, tracing the correct signals on the dismounted ATmega328P, and getting a clean, triggerable power trace took considerably longer than writing the character-recovery logic once a usable signal was available. This matches the course's broader point that physical attacks are often limited less by cryptographic or algorithmic sophistication than by the practical difficulty of instrumenting the target correctly.

The attack also has clear limits. It relies entirely on physical access and dedicated equipment, so it says nothing about remote exploitability. The proposed constant-time counter-measure, in particular, addresses the timing leakage but is not guaranteed to eliminate the power leakage itself, and was never implemented or verified against the actual target, so its effectiveness remains an informed expectation rather than a demonstrated result.

Overall, this project showed that even a deliberately simple embedded system can be made to leak a full secret through a side channel, and that identifying such a vulnerability is often more

[4] `https://cwe.mitre.org/data/definitions/1255.html`

about patient instrumentation than about complex analysis.